

EVIDENCIA DE PRODUCTO

Taller funciones y procedimientos en la solución de algoritmos

GA3-220501093-AA2-EV03

VERSIÓN PÚBLICA ANONIMIZADA

Sin identidad, contacto ni datos académicos individualizantes

10 problemas · 10 pseudocódigos · 10 diagramas de flujo
Funciones, procedimientos, decisiones y ciclos verificables

00 · ARQUITECTURA DE LA SOLUCIÓN

Cómo leer esta entrega

Cada problema se analiza, representa y comprueba con la misma trazabilidad.

Cobertura: La entrega contiene exactamente diez pseudocódigos y diez diagramas. Cada pseudocódigo separa el proceso principal de una función o procedimiento reutilizable.

Método aplicado

FASE	CONTENIDO	RESULTADO VERIFICABLE
Análisis	Enunciado, entradas, proceso, salida, variables y supuestos.	Tabla IPO y casos de prueba.
Diseño	Secuencia, decisiones y ciclos con simbología consistente.	Un diagrama de flujo por problema.
Implementación	Función o procedimiento y proceso principal.	Un archivo PSeInt por problema.
Verificación	Casos normales, frontera y entradas inválidas.	Pruebas de escritorio y pruebas ejecutables.

Simbología

FORMA	SIGNIFICADO	USO EN LOS DIAGRAMAS
Óvalo	Inicio o fin	Delimita cada algoritmo.
Paralelogramo	Entrada o salida	Lectura y presentación de resultados.
Rectángulo	Proceso	Cálculo, asignación o llamada modular.
Rombo	Decisión	Validación, condición o control del ciclo.

Criterios técnicos comunes

- Las entradas inválidas regresan al punto de lectura y no generan resultados parciales.
- Los ciclos tienen inicialización, condición de terminación y actualización visibles.
- La variante pública usa información sintética y metadatos genéricos.

01 · PROBLEMA 1 · ANÁLISIS

Tiempo medio de una maratón

Trazabilidad: análisis p. 3 · diagrama p. 4 · pseudocódigo p. 5.

Enunciado: Un corredor completa 42,195 km en 2 horas y 25 minutos. Calcular el tiempo medio en minutos por kilómetro.

Entradas, proceso y salida

ENTRADAS	PROCESO	SALIDA
Horas, minutos y distancia en kilómetros.	Convertir todo el tiempo a minutos y dividirlo entre la distancia.	Ritmo medio en minutos por kilómetro.

Diseño modular

Módulo: Función CalcularRitmoMinutosPorKilometro(distancia, horas, minutos)

Variables principales

VARIABLE	TIPO	FUNCIÓN
horas / minutos	Entero	Duración de la carrera.
distancia	Real	Recorrido en kilómetros; mayor que cero.
ritmo	Real	Minutos empleados por kilómetro.

Supuestos y restricciones

- La distancia debe ser positiva y los tiempos no pueden ser negativos.
- La función se parametriza aunque el caso de la guía proporcione valores fijos.

Pruebas de escritorio

CASO	ENTRADA	RESULTADO ESPERADO
Caso guía	2 h, 25 min, 42,195 km	3,436426 min/km
Control	0 h, 30 min, 10 km	3 min/km
Inválido	Distancia = 0	Solicitar nuevamente

Diagrama de flujo

Las rutas Sí/No y las flechas de retorno muestran decisiones y repeticiones.

PROBLEMA 01 · TIEMPO MEDIO DE UNA MARATÓN

Módulo principal: Función `CalcularRitmoMinutosPorKilometro(distancia, horas, minutos)`

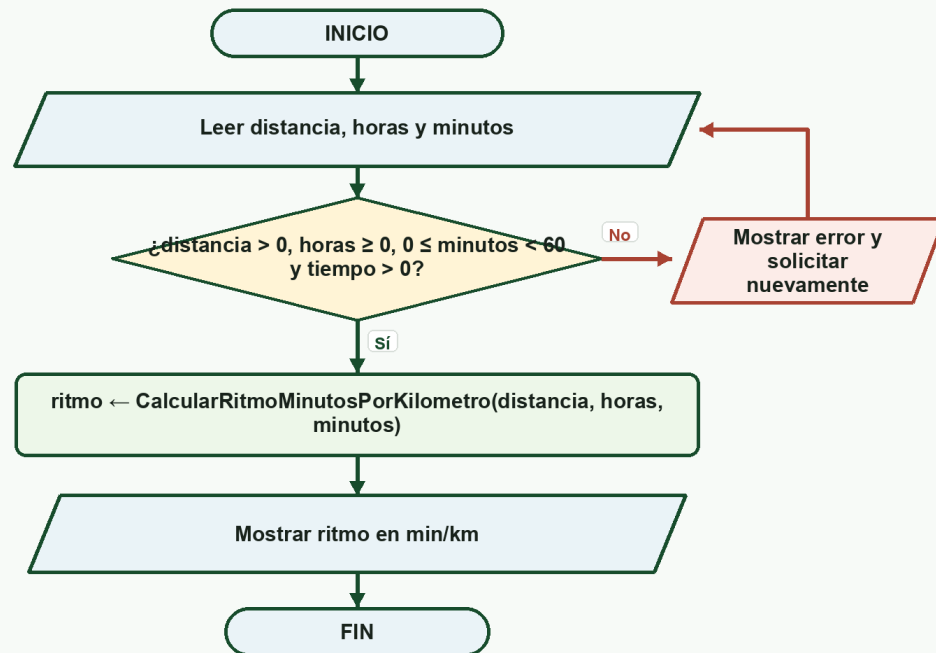


Figura 1. Flujo de tiempo medio de una maratón.

Correspondencia: El diagrama representa la lectura y los controles aplicables, refleja Función `CalcularRitmoMinutosPorKilometro(distancia, horas, minutos)` y conserva la salida descrita en la tabla IPO.

01 · PROBLEMA 1 · IMPLEMENTACIÓN

Pseudocódigo modular

Notación educativa compatible con las construcciones documentadas por PSeInt.

```
// GA3-220501093-AA2-EV03
// Problema 1: calcular el ritmo medio de un maratón de 42.195 km
// recorrido en 2 horas y 25 minutos.
SubProceso valido <- DatosMaratonValidos(distancia, horas, minutos)
  Definir valido Como Logico;
  valido <- distancia > 0 Y horas >= 0 Y minutos >= 0 Y minutos < 60 Y (horas > 0 O minutos > 0);
FinSubProceso
SubProceso ritmo <- CalcularRitmoMinutosPorKilometro(distancia, horas, minutos)
  Definir ritmo, tiempoTotalMinutos Como Real;
  tiempoTotalMinutos <- horas * 60 + minutos;
  ritmo <- tiempoTotalMinutos / distancia;
FinSubProceso
SubProceso MostrarRitmo(ritmo)
  Definir minutosEnteros Como Entero;
  Definir segundos Como Real;
  minutosEnteros <- Trunc(ritmo);
  segundos <- Redon((ritmo - minutosEnteros) * 60 * 100) / 100;
  // Se normaliza un posible redondeo de 60 segundos.
  Si segundos >= 60 Entonces
    minutosEnteros <- minutosEnteros + 1;
    segundos <- 0;
  FinSi
  Escribir "Ritmo medio: ", ritmo, " minutos por kilometro.";
  Escribir "Equivale a ", minutosEnteros, " minutos y ", segundos, " segundos por kilometro.";
FinSubProceso
Proceso RitmoMaraton
  Definir distancia, ritmo Como Real;
  Definir horas, minutos Como Entero;
  Repetir
    Escribir "Ingrese la distancia recorrida en kilometros:";
    Leer distancia;
    Escribir "Ingrese las horas y los minutos empleados:";
    Leer horas, minutos;
    Si NO DatosMaratonValidos(distancia, horas, minutos) Entonces
      Escribir "Datos invalidos. Revise la distancia y el tiempo.";
    FinSi
  Hasta Que DatosMaratonValidos(distancia, horas, minutos)
  ritmo <- CalcularRitmoMinutosPorKilometro(distancia, horas, minutos);
  MostrarRitmo(ritmo);
FinProceso
```

02 · PROBLEMA 2 · ANÁLISIS

Conversión de Celsius a Fahrenheit

Trazabilidad: análisis p. 6 · diagrama p. 7 · pseudocódigo p. 8.

Enunciado: Convertir una temperatura dada en grados Celsius a grados Fahrenheit mediante $F = (9/5)C + 32$.

Entradas, proceso y salida

ENTRADAS	PROCESO	SALIDA
Temperatura real en grados Celsius.	Multiplicar por 9/5 y sumar 32.	Temperatura equivalente en grados Fahrenheit.

Diseño modular

Módulo: Función ConvertirCelsiusAFahrenheit(celsius)

Variables principales

VARIABLE	TIPO	FUNCIÓN
celsius	Real	Temperatura de origen.
fahrenheit	Real	Resultado de la conversión.

Supuestos y restricciones

- Se aceptan temperaturas positivas, negativas y cero.

Pruebas de escritorio

CASO	ENTRADA	RESULTADO ESPERADO
Congelación	0 °C	32 °F
Ebullición	100 °C	212 °F
Coincidencia	-40 °C	-40 °F

Diagrama de flujo

Las rutas Sí/No y las flechas de retorno muestran decisiones y repeticiones.

PROBLEMA 02 · CONVERSIÓN DE CELSIUS A FAHRENHEIT

Módulo principal: Función ConvertirCelsiusAFahrenheit(celsius)

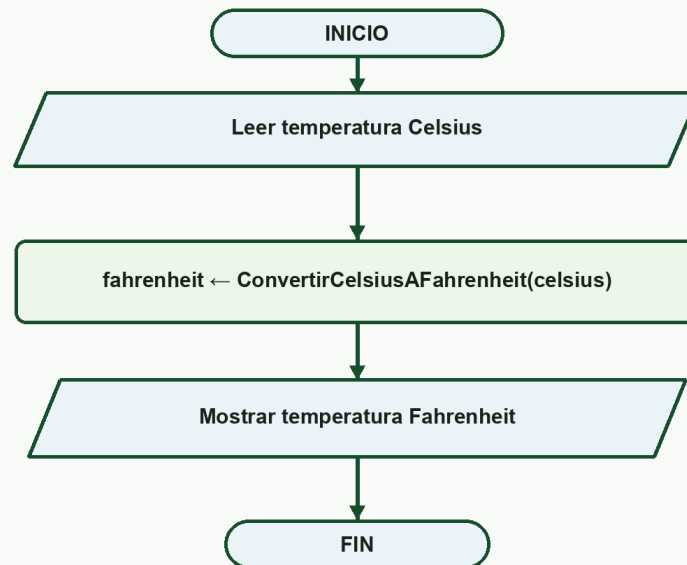


Figura 2. Flujo de conversión de celsius a fahrenheit.

Correspondencia: El diagrama representa la lectura y los controles aplicables, refleja Función ConvertirCelsiusAFahrenheit(celsius) y conserva la salida descrita en la tabla IPO.

02 · PROBLEMA 2 · IMPLEMENTACIÓN

Pseudocódigo modular

Notación educativa compatible con las construcciones documentadas por PSeInt.

```
// GA3-220501093-AA2-EV03
// Problema 2: convertir una temperatura de grados Celsius a Fahrenheit.
SubProceso fahrenheit <- ConvertirCelsiusAFahrenheit(celsius)
  Definir fahrenheit Como Real;
  fahrenheit <- (9.0 / 5.0) * celsius + 32;
FinSubProceso
SubProceso MostrarConversion(celsius, fahrenheit)
  Escribir celsius, " grados Celsius equivalen a ", fahrenheit, " grados Fahrenheit.";
FinSubProceso
Proceso CelsiusAFahrenheit
  Definir celsius, fahrenheit Como Real;
  Escribir "Ingrese la temperatura en grados Celsius:";
  Leer celsius;
  fahrenheit <- ConvertirCelsiusAFahrenheit(celsius);
  MostrarConversion(celsius, fahrenheit);
FinProceso
```


03 · PROBLEMA 3 · ANÁLISIS

Nota del primer parcial

Trazabilidad: análisis p. 9 · diagrama p. 10 · pseudocódigo p. 11.

Enunciado: Calcular la nota del primer parcial: dos talleres y un cuestionario representan conjuntamente el 30 %, y el examen el 70 %.

Entradas, proceso y salida

ENTRADAS	PROCESO	SALIDA
Notas de taller 1, taller 2, cuestionario y examen.	Promediar las tres actividades; ponderar 30 % y sumar el examen ponderado al 70 %.	Nota definitiva del primer parcial.

Diseño modular

Módulo: Función CalcularNotaPrimerParcial(taller1, taller2, quiz, examen)

Variables principales

VARIABLE	TIPO	FUNCIÓN
taller1 / taller2	Real	Notas de los dos talleres.
quiz / examen	Real	Notas del cuestionario y examen.
promedioSeguimiento	Real	Promedio de las tres actividades.
notaFinal	Real	Resultado ponderado.

Supuestos y restricciones

- Se adopta la escala de 0 a 5 para todas las notas.
- Las tres actividades tienen el mismo peso dentro del componente del 30 %.

Pruebas de escritorio

CASO	ENTRADA	RESULTADO ESPERADO
Caso base	4; 3,5; 4,5; 3,8	3,86
Máximo	5; 5; 5; 5	5,00
Inválido	Alguna nota > 5	Solicitar nuevamente

Diagrama de flujo

Las rutas Sí/No y las flechas de retorno muestran decisiones y repeticiones.

PROBLEMA 03 · NOTA DEL PRIMER PARCIAL

Módulo principal: Función CalcularNotaPrimerParcial(taller1, taller2, quiz, examen)

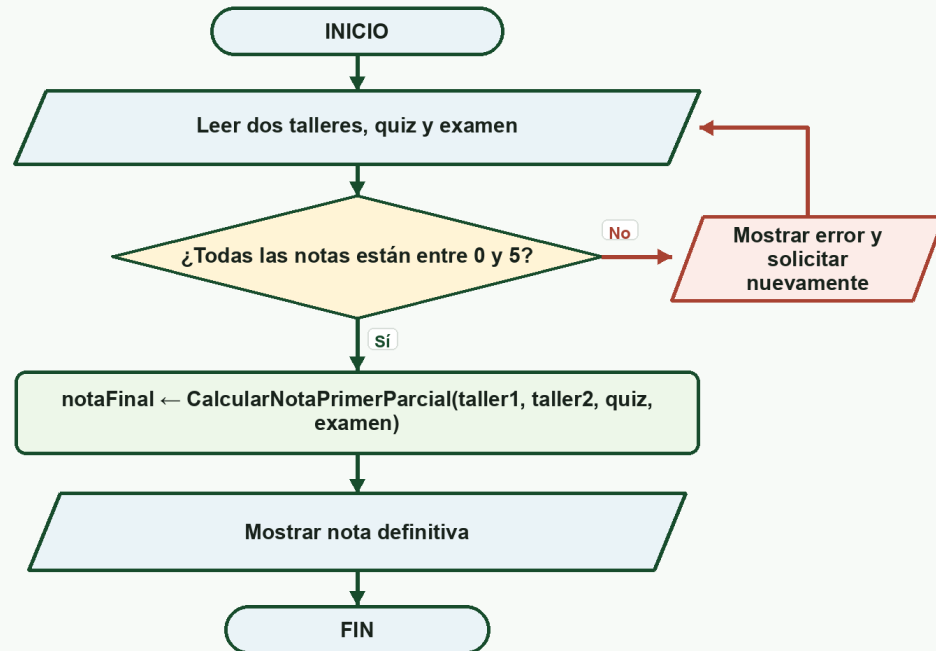


Figura 3. Flujo de nota del primer parcial.

Correspondencia: El diagrama representa la lectura y los controles aplicables, refleja Función CalcularNotaPrimerParcial(taller1, taller2, quiz, examen) y conserva la salida descrita en la tabla IPO.

03 · PROBLEMA 3 · IMPLEMENTACIÓN

Pseudocódigo modular

Notación educativa compatible con las construcciones documentadas por PSeInt.

```
// GA3-220501093-AA2-EV03
// Problema 3: calcular la nota del primer parcial de analisis.
// Supuesto declarado: las dos notas de taller y el quiz tienen el mismo peso
// dentro del componente de seguimiento, que vale el 30 %.
SubProceso valida <- NotaValida(nota)
    Definir valida Como Logico;
    valida <- nota >= 0 Y nota <= 5;
FinSubProceso
SubProceso notaFinal <- CalcularNotaPrimerParcial(taller1, taller2, quiz, examen)
    Definir promedioSeguimiento, notaFinal Como Real;
    promedioSeguimiento <- (taller1 + taller2 + quiz) / 3;
    notaFinal <- promedioSeguimiento * 0.30 + examen * 0.70;
FinSubProceso
Proceso NotaPrimerParcial
    Definir taller1, taller2, quiz, examen, notaFinal Como Real;
    Repetir
        Escribir "Ingrese las notas de taller 1, taller 2, quiz y examen (0 a 5):";
        Leer taller1, taller2, quiz, examen;
        Si NO NotaValida(taller1) O NO NotaValida(taller2) O NO NotaValida(quiz) O NO NotaValida(examen)
            Entonces
                Escribir "Notas invalidas. Todas deben estar entre 0 y 5.";
            FinSi
        Hasta Que NotaValida(taller1) Y NotaValida(taller2) Y NotaValida(quiz) Y NotaValida(examen)
        notaFinal <- CalcularNotaPrimerParcial(taller1, taller2, quiz, examen);
        Escribir "Nota del primer parcial: ", Redon(notaFinal * 100) / 100;
    FinProceso
```

04 · PROBLEMA 4 · ANÁLISIS

Años necesarios para duplicar un capital

Trazabilidad: análisis p. 12 · diagrama p. 13 · pseudocódigo p. 14.

Enunciado: Dado un capital C y una tasa anual R, determinar el primer año en el que el capital alcanza al menos el doble.

Entradas, proceso y salida

ENTRADAS	PROCESO	SALIDA
Capital inicial y tasa anual expresada como porcentaje.	Aplicar capitalización compuesta cada año hasta alcanzar $2 \times C$.	Cantidad entera de años necesarios.

Diseño modular

Módulo: Procedimiento CalcularDuplicacion (salidas por referencia)

Variables principales

VARIABLE	TIPO	FUNCIÓN
capital / meta	Real	Valor inicial y objetivo equivalente al doble.
tasaAnual	Real	Porcentaje anual positivo.
montoFinal	Real	Capital acumulado durante el ciclo.
anios	Entero	Contador de capitalizaciones.

Supuestos y restricciones

- Se usa interés compuesto con capitalización anual.
- Capital y tasa deben ser estrictamente positivos.

Pruebas de escritorio

CASO	ENTRADA	RESULTADO ESPERADO
Caso base	$C = 100000$; $R = 10\%$	8 años
Tasa 100 %	$C = 100$; $R = 100\%$	1 año
Inválido	$R = 0$	Solicitar nuevamente

Diagrama de flujo

Las rutas Sí/No y las flechas de retorno muestran decisiones y repeticiones.

PROBLEMA 04 · AÑOS NECESARIOS PARA DUPLICAR UN CAPITAL

Módulo principal: Procedimiento CalcularDuplicacion (salidas por referencia)

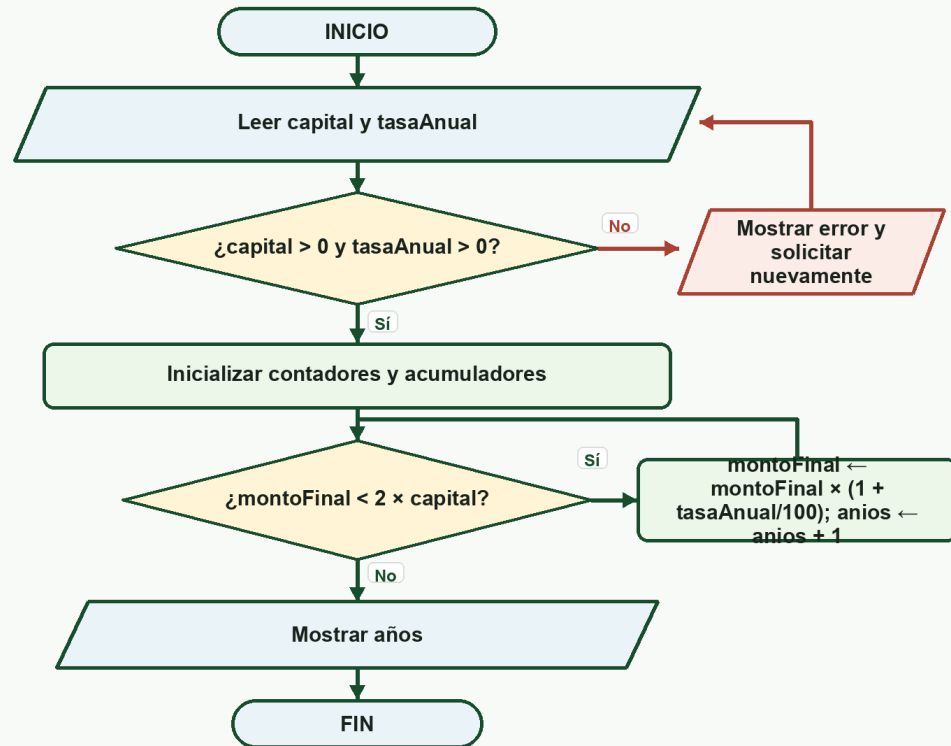


Figura 4. Flujo de años necesarios para duplicar un capital.

Correspondencia: El diagrama representa la lectura y los controles aplicables, refleja Procedimiento CalcularDuplicacion (salidas por referencia) y conserva la salida descrita en la tabla IPO.

04 · PROBLEMA 4 · IMPLEMENTACIÓN

Pseudocódigo modular

Notación educativa compatible con las construcciones documentadas por PSeInt.

```
// GA3-220501093-AA2-EV03
// Problema 4: determinar cuantos años tarda un capital en duplicarse.
// Supuesto declarado: la tasa se expresa como porcentaje anual y los
// intereses se capitalizan una vez al final de cada año.
SubProceso valido <- DatosFinancierosValidos(capital, tasaAnual)
    Definir valido Como Logico;
    valido <- capital > 0 Y tasaAnual > 0;
FinSubProceso
SubProceso CalcularDuplicacion(capital, tasaAnual, años Por Referencia, montoFinal Por Referencia)
    Definir meta Como Real;
    meta <- capital * 2;
    montoFinal <- capital;
    años <- 0;
    Mientras montoFinal < meta Hacer
        montoFinal <- montoFinal * (1 + tasaAnual / 100);
        años <- años + 1;
    FinMientras
FinSubProceso
Proceso DuplicarCapital
    Definir capital, tasaAnual, montoFinal Como Real;
    Definir años Como Entero;
    Repetir
        Escribir "Ingrese el capital inicial, mayor que cero:";
        Leer capital;
        Escribir "Ingrese la tasa de interes anual en porcentaje, mayor que cero:";
        Leer tasaAnual;
        Si NO DatosFinancierosValidos(capital, tasaAnual) Entonces
            Escribir "Datos invalidos. El capital y la tasa deben ser positivos.";
        FinSi
        Hasta Que DatosFinancierosValidos(capital, tasaAnual)
        CalcularDuplicacion(capital, tasaAnual, años, montoFinal);
        Escribir "El capital se duplica al terminar el año ", años, ".";
        Escribir "Monto alcanzado: ", Redon(montoFinal * 100) / 100;
    FinProceso
```

05 · PROBLEMA 5 · ANÁLISIS

Filtrado de veinte números

*Trazabilidad: análisis p. 15 · diagrama p. 16 · pseudocódigo p. 17.***Enunciado:** Ingresar veinte números y mostrar todos los valores menores o iguales a 25.

Entradas, proceso y salida

ENTRADAS	PROCESO	SALIDA
Veinte números reales.	Repetir veinte lecturas; evaluar cada número con una función y mostrarlo si cumple.	Valores menores o iguales a 25 y cantidad encontrada.

Diseño modular

Módulo: Función EsMenorOIgualAlLimite(numero, limite)

Variables principales

VARIABLE	TIPO	FUNCIÓN
numero / limite	Real	Valor leído y frontera fija igual a 25.
posicion / totalDatos	Entero	Controlan las veinte lecturas.
cantidadMostrada	Entero	Cuenta los valores que cumplen.

Supuestos y restricciones

- Se procesan exactamente veinte entradas, incluyendo valores repetidos.

Pruebas de escritorio

CASO	ENTRADA	RESULTADO ESPERADO
Todos cumplen	Valores 1 a 20	Se muestran 20
Caso mixto	Valores 20 a 39	Se muestran 20 a 25
Frontera	Valor = 25	Sí se muestra

Diagrama de flujo

Las rutas Sí/No y las flechas de retorno muestran decisiones y repeticiones.

PROBLEMA 05 · FILTRADO DE VEINTE NÚMEROS

Módulo principal: Función EsMenorOIgualALimite(numero, limite)

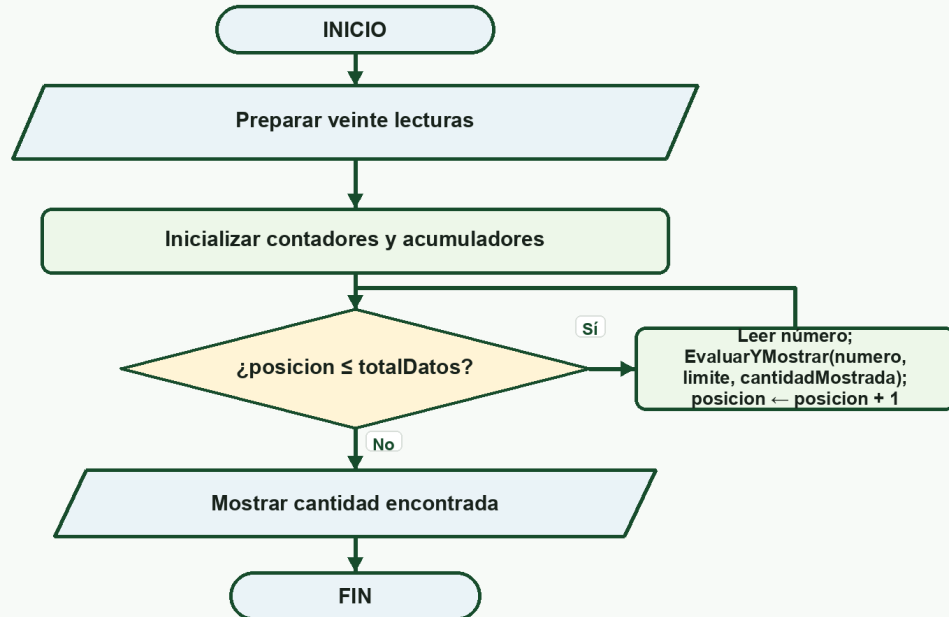


Figura 5. Flujo de filtrado de veinte números.

Correspondencia: El diagrama representa la lectura y los controles aplicables, refleja Función EsMenorOIgualALimite(numero, limite) y conserva la salida descrita en la tabla IPO.

05 · PROBLEMA 5 · IMPLEMENTACIÓN

Pseudocódigo modular

Notación educativa compatible con las construcciones documentadas por PSeInt.

```
// GA3-220501093-AA2-EV03
// Problema 5: ingresar 20 numeros y mostrar los menores o iguales a 25.
SubProceso cumple <- EsMenorOIgualAlLimite(numero, limite)
    Definir cumple Como Logico;
    cumple <- numero <= limite;
FinSubProceso
SubProceso EvaluarYMostrar(numero, limite, cantidadMostrada Por Referencia)
    Si EsMenorOIgualAlLimite(numero, limite) Entonces
        Escribir "Valor que cumple: ", numero;
        cantidadMostrada <- cantidadMostrada + 1;
    FinSi
FinSubProceso
Proceso NumerosMenoresOIgualesA25
    Definir numero, limite Como Real;
    Definir posicion, totalDatos, cantidadMostrada Como Entero;
    totalDatos <- 20;
    limite <- 25;
    cantidadMostrada <- 0;
    Para posicion <- 1 Hasta totalDatos Con Paso 1 Hacer
        Escribir "Ingrese el numero ", posicion, " de ", totalDatos, ":";
        Leer numero;
        EvaluarYMostrar(numero, limite, cantidadMostrada);
    FinPara
    Si cantidadMostrada = 0 Entonces
        Escribir "Ningun numero ingresado es menor o igual a 25.";
    SiNo
        Escribir "Cantidad de numeros mostrados: ", cantidadMostrada;
    FinSi
FinProceso
```

06 · PROBLEMA 6 · ANÁLISIS

Venta de cinco camisas

Trazabilidad: análisis p. 18 · diagrama p. 19 · pseudocódigo p. 20.

Enunciado: Sumar cinco precios de camisas en dólares y mostrar el total de la venta convertido a pesos.

Entradas, proceso y salida

ENTRADAS	PROCESO	SALIDA
Cinco precios en dólares y tasa de cambio en pesos por dólar.	Validar la tasa y los precios, sumar cinco valores y convertir el total.	Total en dólares y total equivalente en pesos.

Diseño modular

Módulo: Función ConvertirDolaresAPesos(totalDolares, tasaCambio)

Variables principales

VARIABLE	TIPO	FUNCIÓN
precio	Real	Precio de cada camisa en dólares.
tasaCambio	Real	Pesos equivalentes a un dólar.
totalDolares / totalPesos	Real	Acumulados antes y después de convertir.
posicion	Entero	Controla cinco lecturas.

Supuestos y restricciones

- La tasa de cambio es una entrada positiva para evitar fijar un valor que cambia con el tiempo.
- Los precios pueden ser cero, pero no negativos.

Pruebas de escritorio

CASO	ENTRADA	RESULTADO ESPERADO
Caso base	20, 25, 30, 15, 10; tasa 4000	USD 100; COP 400000
Precios cero	Cinco ceros; tasa 4100	USD 0; COP 0
Inválido	Tasa ≤ 0	Solicitar nuevamente

Diagrama de flujo

Las rutas Sí/No y las flechas de retorno muestran decisiones y repeticiones.

PROBLEMA 06 · VENTA DE CINCO CAMISAS

Módulo principal: Función ConvertirDolaresAPesos(totalDolares, tasaCambio)

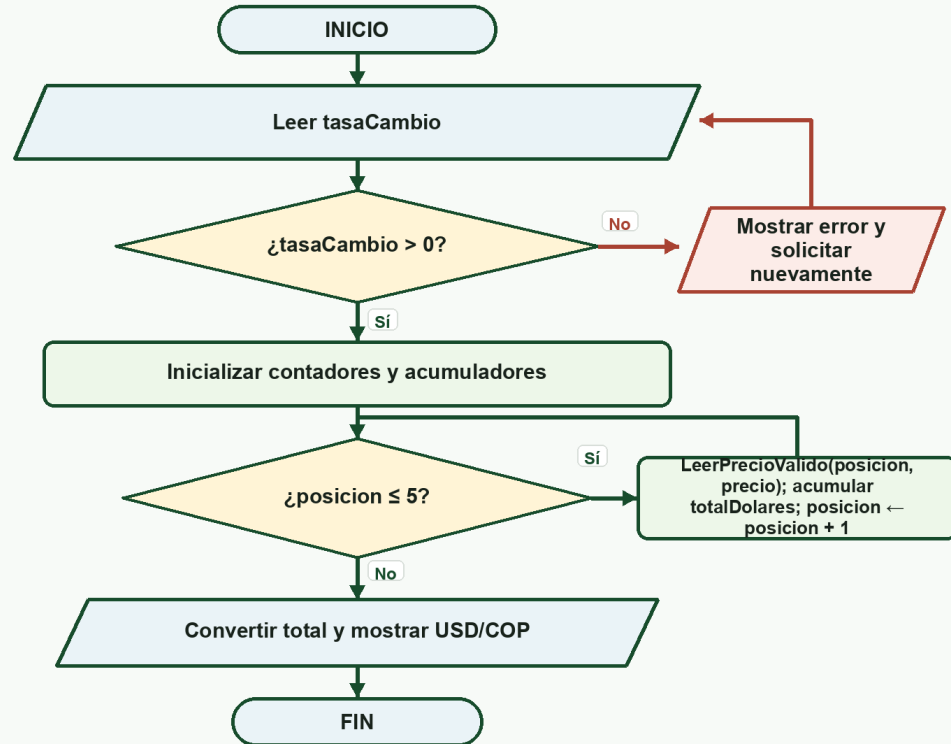


Figura 6. Flujo de venta de cinco camisas.

Correspondencia: El diagrama representa la lectura y los controles aplicables, refleja Función ConvertirDolaresAPesos(totalDolares, tasaCambio) y conserva la salida descrita en la tabla IPO.

06 · PROBLEMA 6 · IMPLEMENTACIÓN

Pseudocódigo modular

Notación educativa compatible con las construcciones documentadas por PSeInt.

```
// GA3-220501093-AA2-EV03
// Problema 6: sumar cinco precios de camisas en dolares y convertir el total a pesos.
// La tasa de cambio se solicita como entrada para no depender de un valor desactualizado.
SubProceso valido <- PrecioValido(precio)
    Definir valido Como Logico;
    valido <- precio >= 0;
FinSubProceso
SubProceso LeerPrecioValido(posicion, precio Por Referencia)
    Repetir
        Escribir "Ingrese el precio en dolares de la camisa ", posicion, ":";
        Leer precio;
        Si NO PrecioValido(precio) Entonces
            Escribir "Precio invalido. No puede ser negativo.";
        FinSi
    Hasta Que PrecioValido(precio)
FinSubProceso
SubProceso totalPesos <- ConvertirDolaresAPesos(totalDolares, tasaCambio)
    Definir totalPesos Como Real;
    totalPesos <- totalDolares * tasaCambio;
FinSubProceso
Proceso CamisasDolaresAPesos
    Definir precio, totalDolares, tasaCambio, totalPesos Como Real;
    Definir posicion Como Entero;
    Repetir
        Escribir "Ingrese cuantos pesos equivalen a un dolar:";
        Leer tasaCambio;
        Si tasaCambio <= 0 Entonces
            Escribir "Tasa invalida. Debe ser mayor que cero.";
        FinSi
    Hasta Que tasaCambio > 0
    totalDolares <- 0;
    Para posicion <- 1 Hasta 5 Con Paso 1 Hacer
        LeerPrecioValido(posicion, precio);
        totalDolares <- totalDolares + precio;
    FinPara
    totalPesos <- ConvertirDolaresAPesos(totalDolares, tasaCambio);
    Escribir "Total en dolares: USD ", Redon(totalDolares * 100) / 100;
    Escribir "Total en pesos: COP ", Redon(totalPesos * 100) / 100;
FinProceso
```

07 · PROBLEMA 7 · ANÁLISIS

Pagos de clientes de un restaurante

Trazabilidad: análisis p. 21 · diagrama p. 22 · pseudocódigo p. 23.

Enunciado: Registrar consumos; aplicar 20 % de descuento cuando cada consumo exceda 50000; mostrar cada pago y el total.

Entradas, proceso y salida

ENTRADAS	PROCESO	SALIDA
Cantidad de clientes y consumo no negativo de cada uno.	Calcular el pago individual, mostrarlo y acumular todos los pagos.	Pago de cada cliente y total general.

Diseño modular

Módulo: Función CalcularPago(consumo)

Variables principales

VARIABLE	TIPO	FUNCIÓN
cantidadClientes	Entero	Número de clientes por procesar.
consumo / pago	Real	Valor original y valor después del descuento.
totalPagos	Real	Acumulador de pagos individuales.
cliente	Entero	Control del recorrido de clientes.

Supuestos y restricciones

- La cantidad de clientes se solicita antes de iniciar y debe ser positiva.
- Un consumo exactamente igual a 50000 no recibe descuento.

Pruebas de escritorio

CASO	ENTRADA	RESULTADO ESPERADO
Frontera	Consumo 50000	Pago 50000
Con descuento	Consumo 60000	Pago 48000
Total	Consumos 50000 y 60000	Total 98000

Diagrama de flujo

Las rutas Sí/No y las flechas de retorno muestran decisiones y repeticiones.

PROBLEMA 07 · PAGOS DE CLIENTES DE UN RESTAURANTE

Módulo principal: Función CalcularPago(consumo)

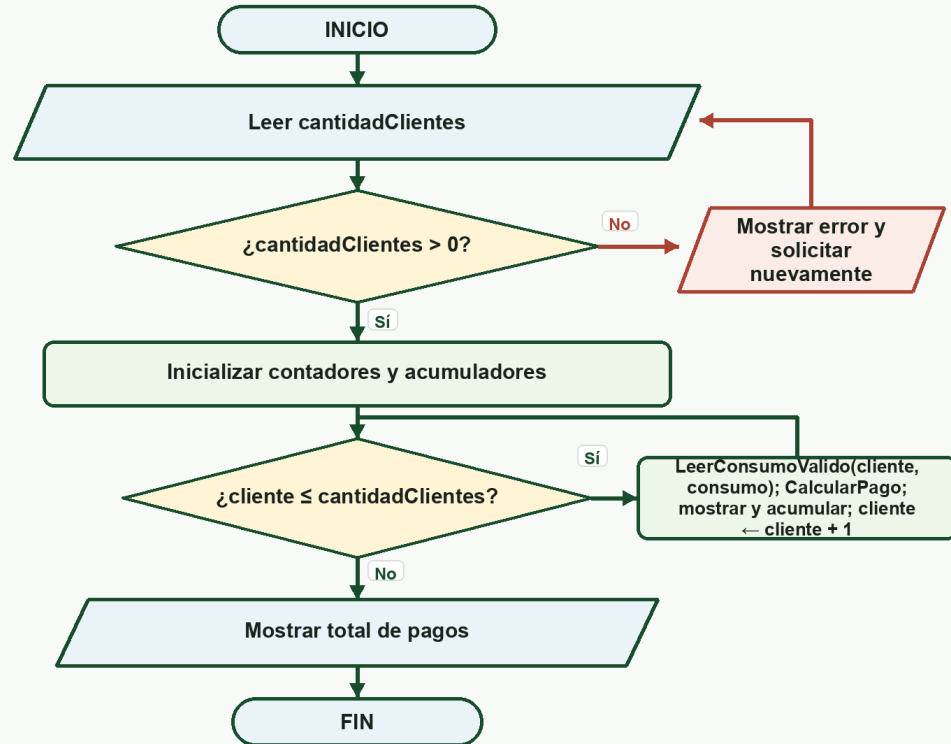


Figura 7. Flujo de pagos de clientes de un restaurante.

Correspondencia: El diagrama representa la lectura y los controles aplicables, refleja Función CalcularPago(consumo) y conserva la salida descrita en la tabla IPO.

07 · PROBLEMA 7 · IMPLEMENTACIÓN

Pseudocódigo modular

Notación educativa compatible con las construcciones documentadas por PSeInt.

```
// GA3-220501093-AA2-EV03
// Problema 7: calcular los pagos de clientes de un restaurante.
// El descuento del 20 % se aplica solo cuando el consumo excede 50000.
SubProceso valido <- ConsumoValido(consumo)
    Definir valido Como Logico;
    valido <- consumo >= 0;
FinSubProceso
SubProceso pago <- CalcularPago(consumo)
    Definir pago Como Real;
    Si consumo > 50000 Entonces
        pago <- consumo * 0.80;
    SiNo
        pago <- consumo;
    FinSi
FinSubProceso
SubProceso LeerConsumoValido(numeroCliente, consumo Por Referencia)
    Repetir
        Escribir "Ingrese el consumo del cliente ", numeroCliente, ":";
        Leer consumo;
        Si NO ConsumoValido(consumo) Entonces
            Escribir "Consumo invalido. No puede ser negativo.";
        FinSi
    Hasta Que ConsumoValido(consumo)
FinSubProceso
Proceso ConsumosRestaurante
    Definir cantidadClientes, cliente Como Entero;
    Definir consumo, pago, totalPagos Como Real;
    Repetir
        Escribir "Ingrese la cantidad de clientes, mayor que cero:";
        Leer cantidadClientes;
        Si cantidadClientes <= 0 Entonces
            Escribir "Cantidad invalida. Debe existir al menos un cliente.";
        FinSi
    Hasta Que cantidadClientes > 0
    totalPagos <- 0;
    Para cliente <- 1 Hasta cantidadClientes Con Paso 1 Hacer
        LeerConsumoValido(cliente, consumo);
        pago <- CalcularPago(consumo);
        totalPagos <- totalPagos + pago;
        Si consumo > 50000 Entonces
            Escribir "Cliente ", cliente, ": pago con descuento = ", Redon(pago * 100) / 100;
        SiNo
            Escribir "Cliente ", cliente, ": pago sin descuento = ", Redon(pago * 100) / 100;
        FinSi
    FinPara
    Escribir "Total de todos los pagos: ", Redon(totalPagos * 100) / 100;
FinProceso
```

08 · PROBLEMA 8 · ANÁLISIS

Hora del siguiente segundo

Trazabilidad: análisis p. 24 · diagrama p. 25 · pseudocódigo p. 26.

Enunciado: Ingresar hora, minutos y segundos válidos y calcular la hora correspondiente al siguiente segundo.

Entradas, proceso y salida

ENTRADAS	PROCESO	SALIDA
Hora de 0 a 23; minutos y segundos de 0 a 59.	Incrementar segundos y propagar los cambios a minutos y horas, incluyendo el cambio de día.	Hora, minuto y segundo posteriores.

Diseño modular

Módulo: Procedimiento AvanzarUnSegundo (parámetros por referencia)

Variables principales

VARIABLE	TIPO	FUNCIÓN
hora	Entero	Componente horaria; rango 0..23.
minuto / segundo	Entero	Componentes; rango 0..59.
valida	Lógico	Controla la aceptación de la entrada.

Supuestos y restricciones

- Después de 23:59:59 se obtiene 00:00:00.

Pruebas de escritorio

CASO	ENTRADA	RESULTADO ESPERADO
Normal	12:30:45	12:30:46
Cambio de hora	12:59:59	13:00:00
Cambio de día	23:59:59	00:00:00

Diagrama de flujo

Las rutas Sí/No y las flechas de retorno muestran decisiones y repeticiones.

PROBLEMA 08 · HORA DEL SIGUIENTE SEGUNDO

Módulo principal: Procedimiento AvanzarUnSegundo (parámetros por referencia)

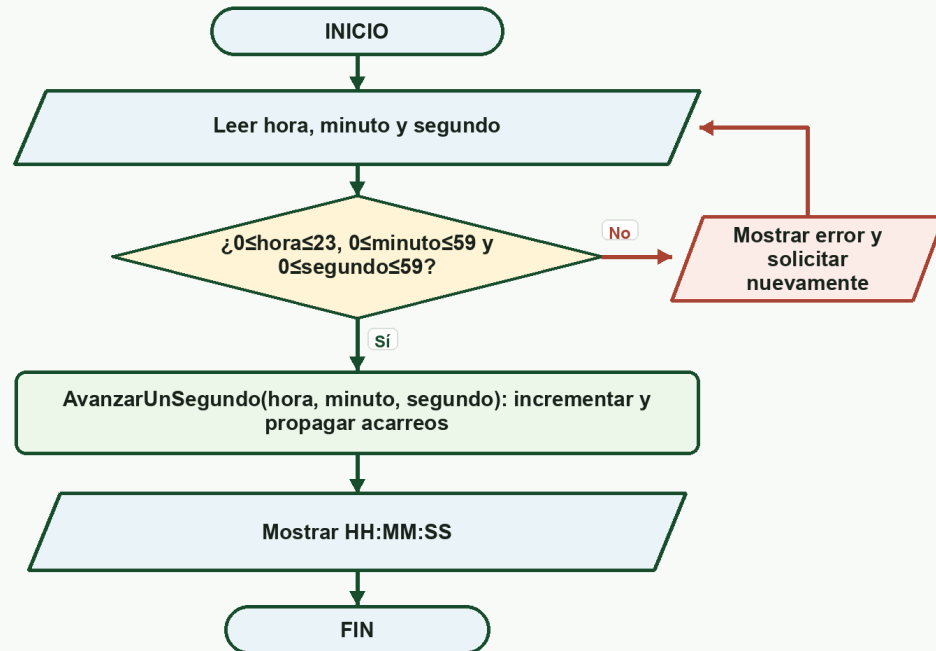


Figura 8. Flujo de hora del siguiente segundo.

Correspondencia: El diagrama representa la lectura y los controles aplicables, refleja Procedimiento AvanzarUnSegundo (parámetros por referencia) y conserva la salida descrita en la tabla IPO.

08 · PROBLEMA 8 · IMPLEMENTACIÓN

Pseudocódigo modular

Notación educativa compatible con las construcciones documentadas por PSeInt.

```
// GA3-220501093-AA2-EV03
// Problema 8: calcular la hora correspondiente al siguiente segundo.
SubProceso valida <- HoraValida(hora, minuto, segundo)
    Definir valida Como Logico;
    valida <- hora >= 0 Y hora <= 23 Y minuto >= 0 Y minuto <= 59 Y segundo >= 0 Y segundo <= 59;
FinSubProceso
SubProceso AvanzarUnSegundo(hora Por Referencia, minuto Por Referencia, segundo Por Referencia)
    segundo <- segundo + 1;
    Si segundo = 60 Entonces
        segundo <- 0;
        minuto <- minuto + 1;
        Si minuto = 60 Entonces
            minuto <- 0;
            hora <- hora + 1;
            Si hora = 24 Entonces
                hora <- 0;
            FinSi
        FinSi
    FinSi
FinSubProceso
Proceso SiguienteSegundo
    Definir hora, minuto, segundo Como Entero;
    Repetir
        Escribir "Ingrese la hora, los minutos y los segundos:";
        Leer hora, minuto, segundo;
        Si NO HoraValida(hora, minuto, segundo) Entonces
            Escribir "Hora invalida. Use H=0..23, M=0..59 y S=0..59.";
        FinSi
    Hasta Que HoraValida(hora, minuto, segundo)
    AvanzarUnSegundo(hora, minuto, segundo);
    Escribir "Hora en el siguiente segundo: ", hora, ":", minuto, ":", segundo;
FinProceso
```

09 · PROBLEMA 9 · ANÁLISIS

Producto desde 1 hasta N

*Trazabilidad: análisis p. 27 · diagrama p. 28 · pseudocódigo p. 29.***Enunciado:** Dado un número entero N, calcular el producto de todos los enteros desde 1 hasta N.

Entradas, proceso y salida

ENTRADAS	PROCESO	SALIDA
Número entero N mayor o igual a cero.	Inicializar el producto en 1 y multiplicar sucesivamente desde 1 hasta N.	Producto acumulado, equivalente a N factorial.

Diseño modular

Módulo: Función CalcularProductoUnoHastaN(n)

Variables principales

VARIABLE	TIPO	FUNCIÓN
numeroIngresado / producto	Real	Dato leído y acumulador inicializado en 1.
n / factor	Entero	Límite validado y contador desde 1.

Supuestos y restricciones

- Se acepta $N = 0$ y se aplica la convención $0! = 1$.
- Para evitar desbordamiento se recomienda trabajar con valores pequeños.

Pruebas de escritorio

CASO	ENTRADA	RESULTADO ESPERADO
Caso base	$N = 5$	120
Caso cero	$N = 0$	1
Inválido	$N < 0$	Solicitar nuevamente

Diagrama de flujo

Las rutas Sí/No y las flechas de retorno muestran decisiones y repeticiones.

PROBLEMA 09 · PRODUCTO DESDE 1 HASTA N

Módulo principal: Función CalcularProductoUnoHastaN(n)

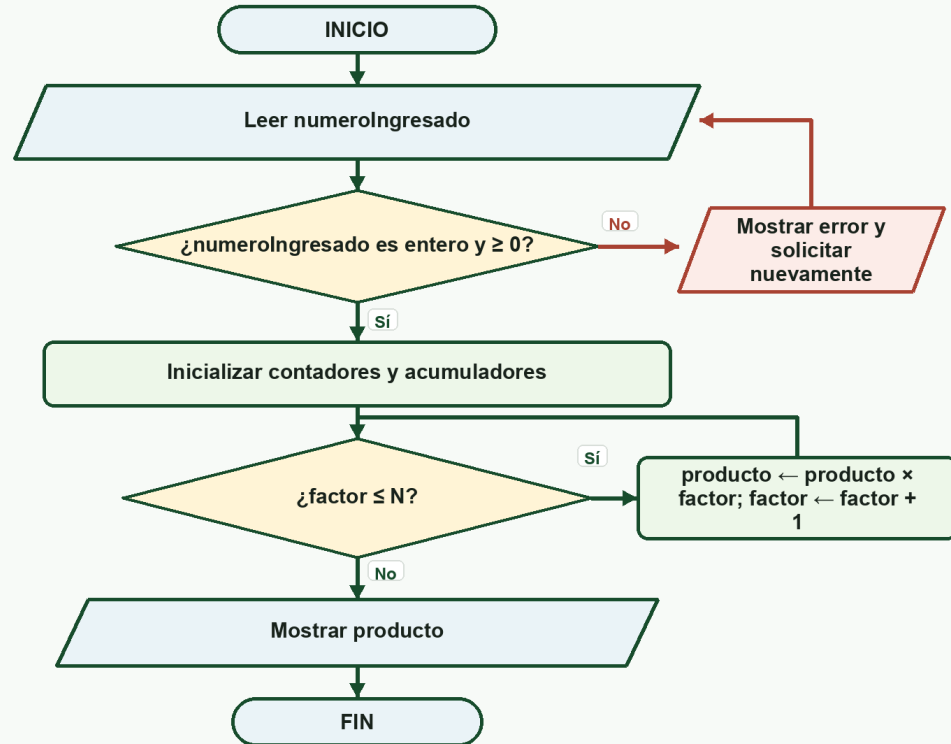


Figura 9. Flujo de producto desde 1 hasta n.

Correspondencia: El diagrama representa la lectura y los controles aplicables, refleja Función CalcularProductoUnoHastaN(n) y conserva la salida descrita en la tabla IPO.

09 · PROBLEMA 9 · IMPLEMENTACIÓN

Pseudocódigo modular

Notación educativa compatible con las construcciones documentadas por PSeInt.

```
// GA3-220501093-AA2-EV03
// Problema 9: calcular el producto de los enteros desde 1 hasta N.
// Se admite N = 0 y se aplica la convencion de producto vacio: 0! = 1.
SubProceso valido <- EsEnteroNoNegativo(numero)
  Definir valido Como Logico;
  valido <- numero >= 0 Y numero = Trunc(numero);
FinSubProceso
SubProceso producto <- CalcularProductoUnoHastaN(n)
  Definir producto Como Real;
  Definir factor Como Entero;
  producto <- 1;
  Para factor <- 1 Hasta n Con Paso 1 Hacer
    producto <- producto * factor;
  FinPara
FinSubProceso
Proceso ProductoUnoHastaN
  Definir numeroIngresado, producto Como Real;
  Definir n Como Entero;
  Repetir
    Escribir "Ingrese un numero entero no negativo:";
    Leer numeroIngresado;
    Si NO EsEnteroNoNegativo(numeroIngresado) Entonces
      Escribir "Dato invalido. Debe ser un entero mayor o igual a cero.";
    FinSi
  Hasta Que EsEnteroNoNegativo(numeroIngresado)
  n <- Trunc(numeroIngresado);
  producto <- CalcularProductoUnoHastaN(n);
  Escribir "El producto desde 1 hasta ", n, " es ", producto, ".";
FinProceso
```

10 · PROBLEMA 10 · ANÁLISIS

Tabla de multiplicar decreciente

Trazabilidad: análisis p. 30 · diagrama p. 31 · pseudocódigo p. 32.

Enunciado: Mostrar la tabla de multiplicar decreciente de un número ingresado entre 1 y 10.

Entradas, proceso y salida

ENTRADAS	PROCESO	SALIDA
Número entero en el rango de 1 a 10.	Recorrer los multiplicadores desde 10 hasta 1 y mostrar cada producto.	Diez filas de la tabla en orden descendente.

Diseño modular

Módulo: Procedimiento MostrarTablaDecreciente(numero)

Variables principales

VARIABLE	TIPO	FUNCIÓN
numeroIngresado	Real	Dato leído antes de validar su integralidad.
numero	Entero	Base de la tabla; rango 1..10.
multiplicador	Entero	Contador descendente desde 10.

Supuestos y restricciones

- El multiplicador disminuye de uno en uno: 10, 9, ..., 1.

Pruebas de escritorio

CASO	ENTRADA	RESULTADO ESPERADO
Primera fila	Número 3; factor 10	$3 \times 10 = 30$
Última fila	Número 3; factor 1	$3 \times 1 = 3$
Inválido	Número 11	Solicitar nuevamente

10 · PROBLEMA 10 · DISEÑO

Diagrama de flujo

Las rutas Sí/No y las flechas de retorno muestran decisiones y repeticiones.

PROBLEMA 10 · TABLA DE MULTIPLICAR DECRECIENTE

Módulo principal: Procedimiento MostrarTablaDecreciente(numero)

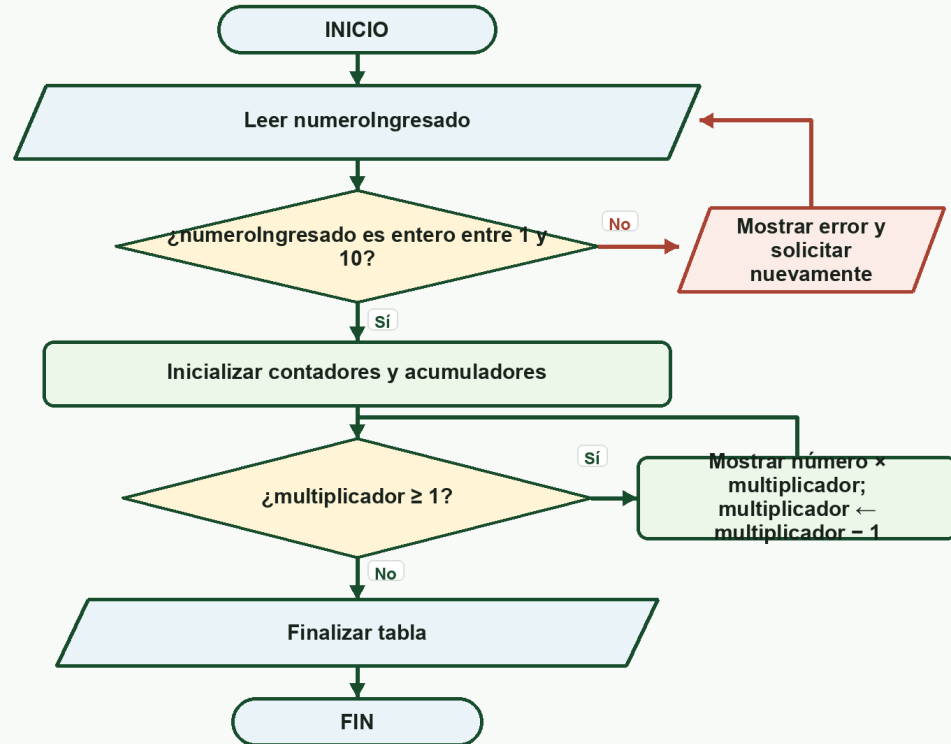


Figura 10. Flujo de tabla de multiplicar decreciente.

Correspondencia: El diagrama representa la lectura y los controles aplicables, refleja Procedimiento MostrarTablaDecreciente(numero) y conserva la salida descrita en la tabla IPO.

10 · PROBLEMA 10 · IMPLEMENTACIÓN

Pseudocódigo modular

Notación educativa compatible con las construcciones documentadas por PSeInt.

```
// GA3-220501093-AA2-EV03
// Problema 10: mostrar la tabla de multiplicar en orden decreciente.
SubProceso valido <- NumeroDeTablaValido(numero)
    Definir valido Como Logico;
    valido <- numero >= 1 Y numero <= 10 Y numero = Trunc(numero);
FinSubProceso
SubProceso MostrarTablaDecreciente(numero)
    Definir multiplicador Como Entero;
    Para multiplicador <- 10 Hasta 1 Con Paso -1 Hacer
        Escribir numero, " x ", multiplicador, " = ", numero * multiplicador;
    FinPara
FinSubProceso
Proceso TablaMultiplicarDecreciente
    Definir numeroIngresado Como Real;
    Definir numero Como Entero;
    Repetir
        Escribir "Ingrese un numero entero entre 1 y 10:";
        Leer numeroIngresado;
        Si NO NumeroDeTablaValido(numeroIngresado) Entonces
            Escribir "Dato invalido. Debe ser un entero entre 1 y 10.";
        FinSi
    Hasta Que NumeroDeTablaValido(numeroIngresado)
    numero <- Trunc(numeroIngresado);
    MostrarTablaDecreciente(numero);
FinProceso
```


11 · CONTROL DE CALIDAD

Cobertura de la lista de chequeo

Cada fila corresponde a dos indicadores binarios del instrumento original.

PROBLEMA	NOTACIÓN DE DIAGRAMA	NOTACIÓN DE PSEUDOCÓDIGO
Problema 1	Diagrama completo — p. 4	Pseudocódigo completo — p. 5
Problema 2	Diagrama completo — p. 7	Pseudocódigo completo — p. 8
Problema 3	Diagrama completo — p. 10	Pseudocódigo completo — p. 11
Problema 4	Diagrama completo — p. 13	Pseudocódigo completo — p. 14
Problema 5	Diagrama completo — p. 16	Pseudocódigo completo — p. 17
Problema 6	Diagrama completo — p. 19	Pseudocódigo completo — p. 20
Problema 7	Diagrama completo — p. 22	Pseudocódigo completo — p. 23
Problema 8	Diagrama completo — p. 25	Pseudocódigo completo — p. 26
Problema 9	Diagrama completo — p. 28	Pseudocódigo completo — p. 29
Problema 10	Diagrama completo — p. 31	Pseudocódigo completo — p. 32

Resultado de las comprobaciones

- Las implementaciones de referencia ejecutan casos normales, límites y rechazos de entradas inválidas.
- El ZIP público contiene los diez pseudocódigos, los diez diagramas, el DOCX y un archivo de lectura.
- El informe personalizado se genera fuera del repositorio y no forma parte de la publicación.

Conclusiones

- La programación modular reduce repeticiones y concentra cada regla de negocio en una función o procedimiento.
- Las validaciones impiden cálculos indefinidos y hacen explícitos los rangos aceptados.
- La equivalencia entre tabla IPO, diagrama, pseudocódigo y prueba facilita revisar cada solución.

Fuentes

- Guía de aprendizaje GA3, páginas 5–6: enunciados y lineamientos de entrega.
- Material oficial Análisis y solución de problemas aplicando algoritmos.
- Documentación oficial de subprocesos de PSeInt.